IT Deployment Guide — resiDNA Streamlit App

Handoff notes for making this app privately and securely reachable inside the company, instead of the Demo Streamlit Community Cloud link. Two realistic paths are below: pure on-prem/local hosting, and hosting on Azure with the app embedded in SharePoint. Both need the same core hardening; SharePoint just changes where users click to get there.

Decided: Option A (on-prem/local server). For the concrete, from-scratch execution of the steps below — actual commands, service files, and config templates, written for someone starting with nothing set up — see IT_Setup_Guide_OptionA.md. Option B stays documented here for reference in case that decision is revisited.

What's already true about the app (no changes needed here)

- `app.py` processes uploaded files entirely in memory — parses with pandas, renders Plotly charts, builds the PDF in memory for download. By default, nothing is written to disk or a database on the server, and nothing persists once a session ends.
- Exception: an opt-in audit log (`app/audit_log.py`), off unless the `RESIDNA_AUDIT_LOG` environment variable is set — see Option A step 7 below. Off by default means the Community Cloud demo's documented behavior (nothing persisted) is unaffected unless someone deliberately enables it there too.
- The app itself has no authentication, no TLS, and no persistent-service setup — all three need to be added by hosting infrastructure, not the app code.

Option A — On-Prem / Local Server Hosting

1. Host on a company-controlled server or VM on the internal network — not exposed to the public internet.
2. Run it as a persistent service, not a terminal session (today, closing the terminal or rebooting stops the app):
   - Linux: a `systemd` unit, or a Docker container with a restart policy.
   - Windows: an NSSM-wrapped service, a Windows container, or Task Scheduler running at startup.
3. Put a reverse proxy in front (Nginx, IIS, or the company's existing load balancer) to:
   - Terminate TLS with a company-issued certificate, so the internal URL is `https://`.
   - Sit between users and the Streamlit process so raw/unauthenticated traffic never reaches it directly.
4. Add authentication — Streamlit has native login support for this: `st.login()` with an `[auth]` block in `.streamlit/secrets.toml`, using OpenID Connect. It works with any OIDC provider, including Microsoft Entra ID, so employees can log in with their existing company account rather than a separate password. (Streamlit docs)
   - Alternative: an auth proxy in front of the app (e.g. oauth2-proxy) if IT prefers to keep auth entirely outside the app.
5. Restrict network reachability: bind to an internal DNS name/IP only, gate with firewall rules or VPN-only access — no public inbound port.
6. Dependency versions are pinned: `app/requirements.txt` lists exact tested versions — reproduce the environment from it as-is and run it through standard vulnerability scanning before go-live.
7. Enable the audit log: compliance has confirmed an audit trail is required, and it's built — `app/audit_log.py` appends one record per generated PDF report (timestamp, uploaded filename, a SHA-256 hash of it, assay/run info, submitter/reviewer name, and pass/fail — never the actual analytical results) to `Logs/audit.log`. It's off unless the `RESIDNA_AUDIT_LOG` environment variable is set on the server process — see IT_Setup_Guide_OptionA.md steps 4–6 for the exact service config and log retention/rotation setup.

Option B — Azure Hosting + SharePoint Embed

SharePoint itself cannot run a Python/Streamlit backend — there's no application runtime inside SharePoint Online for that. What is realistic:

1. Host the app on infrastructure your Microsoft tenant controls — Azure App Service (Linux, native Python support) or Azure Container Apps, inside your tenant's network/VNet.
2. Enable Entra ID authentication on the Azure side — either Azure App Service's built-in authentication ("Easy Auth") or the app's own `st.login()` OIDC flow pointed at your Entra tenant — so it matches company SSO.
3. Embed the hosted app in a SharePoint page using the built-in Embed web part (an iframe pointed at the internal Azure URL). This gives users the experience of it living inside SharePoint, while Azure remains the actual security boundary.
4. Configure the Azure-hosted app to allow being framed by your SharePoint domain. SharePoint Online enforces a Content-Security-Policy that blocks framing from arbitrary external origins. The fix is on the app's hosting config — set a `Content-Security-Policy: frame-ancestors` header (or equivalent App Service setting) that explicitly allows `https://yourtenant.sharepoint.com`. There is no Entra ID or SharePoint-side setting that bypasses this — it has to be granted by the app's own host.
5. Steps 5–7 from Option A (network restriction scope, dependency pinning, logging/audit policy) still apply — embedding in SharePoint is a UI convenience, not a substitute for the same hardening.

Before either option goes live

- Keep `residna.streamlit.app` (Community Cloud) as a demo-only URL — use the company-hosted deployment for real data once it's live.
- `app/requirements.txt` is already pinned — build whichever environment from it as-is so it matches what's been tested.

Sources

- User authentication and information — Streamlit Docs
- st.login — Streamlit Docs
- Embed SharePoint / CSP frame-ancestors discussion — Microsoft Q&A